Лабораторна робота №4. ASP.NET Core Razor Pages: відгуки, звернення та відповіді адміністратора

1. Що ми будуємо і навіщо

У попередніх роботах було створено каталог з реєстрацією, входом, відгуками та адміністративним керуванням записами. У четвертій роботі цей застосунок отримує засоби спілкування між користувачем і адміністрацією.

Розглядаються два різні сценарії:

Сценарій	Де користувач залишає повідомлення	Де з'являється відповідь
Відгук про об'єкт каталогу	На сторінці конкретного фільму, книги тощо	На сайті, поруч із відгуком
Звернення до адміністрації	У формі «Контактна інформація»	В електронній пошті користувача; копія зберігається для адміністратора

Відгук пов'язаний із конкретним об'єктом каталогу. Звернення може стосуватися роботи всього сайту: помилки, пропозиції, прохання про допомогу.

Ключова відмінність: зберегти текст у базі даних і надіслати лист — це дві окремі дії. Одна не відбувається автоматично внаслідок іншої.

У конспекті використано назви з лабораторної: `RazorPagesMovie`, `Review`, `Feedback`, `RazorPagesMovieContext`. Якщо ваш каталог має іншу тематику, назви предметних сутностей можуть відрізнятися. Наведені фрагменти пояснюють окремі механізми, а не замінюють повний код роботи.

2. Як взаємодіють частини застосунку

```
Браузер: користувач заповнює форму
                  ↓ POST-запит
PageModel: отримує та перевіряє введення
                  ↓
DbContext: читає або зберігає дані
                  ↓
База даних: містить відгуки, звернення та відповіді

Для відповіді на звернення додатково:
PageModel → IEmailSender → SMTP-сервер → пошта адресата
```

Браузер не звертається безпосередньо до бази даних і не повинен знати пароль SMTP-сервера. Ці операції виконує серверна частина застосунку.

Основні поняття

Поняття	Просте пояснення	Приклад у роботі
Razor Page	Сторінка з HTML і вставками Razor	`Reply.cshtml`
PageModel	Клас, який готує дані та обробляє запити сторінки	`ReplyModel` у `Reply.cshtml.cs`
Сутність	Об'єкт, дані якого EF Core зберігає в БД	`Review`, `Feedback`
Модель введення	Набір полів, які дозволено отримати з форми	`ContactInput`, `ReplyInput`
Модель відображення	Набір даних, потрібних для показу	`RowVM` для рядка таблиці
Сервіс	Клас з окремою прикладною функцією	Надсилання електронних листів

Слово «модель» використовується в кількох значеннях. `Feedback` — це не те саме, що `IndexModel`: перший описує звернення, другий — поведінку сторінки зі зверненнями.

3. Де розміщуються файли

Нижче — структура основних частин після фінального перенесення адміністративних сторінок. Це не повний перелік файлів проєкту: наявні сторінки Identity, файли налаштувань та інші матеріали залишаються.

```
RazorPagesMovie/
├── Program.cs
├── appsettings.json
├── Models/
│   ├── Movie.cs
│   ├── Review.cs
│   └── Feedback.cs
├── Data/
│   └── RazorPagesMovieContext.cs
├── Migrations/
│   └── ... файли змін структури БД
├── Services/
│   ├── EmailOptions.cs
│   ├── IEmailSender.cs
│   ├── SmtpEmailSender.cs
│   └── ConsoleEmailSender.cs
├── Pages/
│   ├── Index.cshtml
│   ├── Index.cshtml.cs
```

```
│   ├── Contact/
│   │   ├── Index.cshtml
│   │   └── Index.cshtml.cs
│   ├── Catalog/
│   │   └── ... публічні сторінки каталогу
│   ├── Admin/
│   │   ├── Movies/
│   │   │   └── ... сторінки керування каталогом
│   │   ├── Reviews/
│   │   │   ├── Index.cshtml
│   │   │   ├── Index.cshtml.cs
│   │   │   ├── Reply.cshtml
│   │   │   └── Reply.cshtml.cs
│   │   └── Feedback/
│   │       ├── Index.cshtml
│   │       ├── Index.cshtml.cs
│   │       ├── Reply.cshtml
│   │       └── Reply.cshtml.cs
│   └── Shared/
│       ├── _Layout.cshtml
│       └── _ValidationScriptsPartial.cshtml
└── wwwroot/
    └── ... стилі, скрипти та зображення
```

Якщо контекст у вашому попередньому проєкті розміщений не в `Data`, використовуйте його фактичне розташування. Створювати другий контекст лише заради цієї схеми не потрібно.

У початкових кроках методички `Movies` і `Reviews` ще лежать безпосередньо в `Pages`. Перенесення в `Pages/Admin` — окреме фінальне завдання.

Папка визначає стандартний шлях сторінки, а namespace — повне ім'я C#-класу. Написати `Pages.Admin.Reviews` у namespace недостатньо, щоб сторінка отримала адресу `/Admin/Reviews`.

4. Розширення відгуку: одна відповідь адміністратора

До `Review` додаються властивості:

```
[StringLength(2000)]
public string? AdminReply { get; set; }

[StringLength(128)]
public string? AdminReplyBy { get; set; }

public DateTime? AdminRepliedAt { get; set; }
```

- `AdminReply` — текст відповіді.
- `AdminReplyBy` — ім'я або email адміністратора на момент відповіді.
- `AdminRepliedAt` — час відповіді.

Знак `?` означає можливість відсутності значення: `null`. Для `DateTime?` це nullable-тип значення, а для `string?` — позначення допустимості `null` для посилання, яке також враховують аналізатор C#

та домовленості EF Core.

Новий відгук ще не має відповіді, тому ці поля можуть бути порожніми. Не потрібно вигадувати дату або зберігати текст «Відповіді немає» в базі: це повідомлення можна показати в інтерфейсі.

`[StringLength(2000)]` описує обмеження довжини. Але сам атрибут не замінює перевірку `ModelState` у коді обробника форми.

У цій структурі відгук має **одну поточну відповідь**. Повторне збереження змінює її, а не додає нове повідомлення в діалог. Для історії листування знадобилася б окрема колекція відповідей.

Короткий розбір синтаксису

```
var reply = string.IsNullOrWhiteSpace(Input.AdminReply)
    ? null
    : Input.AdminReply.Trim();
```

`var` просить компілятор визначити тип змінної. `IsNullOrWhiteSpace` перевіряє, чи немає змістовного тексту. Конструкція `умова ? значення1 : значення2` вибирає один із двох результатів. `Trim()` прибирає пробіли на початку та в кінці.

Тут порожня відповідь перетворюється на `null`. Якщо адміністратор видаляє відповідь, доцільно також очистити її автора та дату, як і зроблено в методичці.

Оператор `??` має інше призначення:

```
var name = user?.Email ?? user?.UserName ?? "admin";
```

Вибирається перше значення, яке не є `null`. `?.` дозволяє звернутися до властивості лише тоді, коли об’єкт існує. Важливо: `??` не вважає порожній рядок `""` відсутнім значенням.

5. Навіщо потрібні DbContext і міграції

Зміна класу в редакторі ще не означає, що в наявній базі з’явилися нові стовпці.

`RazorPagesMovieContext` — об’єкт для роботи EF Core з даними застосунку. Наприклад:

```
public DbSet<Feedback> Feedbacks { get; set; } = default!;
```

`DbSet<Feedback>` дає змогу формувати запити до звернень, додавати їх і видаляти. Це не просто список, який містить усі записи в оперативній пам’яті.

Позначення `default!` приглушує попередження аналізатора про початкове значення: цю властивість має ініціалізувати EF Core. Воно не завантажує записи з БД.

Міграція — опис зміни структури бази. EF Core порівнює модель зі збереженим знімком попередньої моделі й генерує зміни. Створення міграції та застосування її до БД — окремі кроки.

Документація про міграції.

Для нової сутності `Feedback` у звичайному терміналі, у каталозі проєкту:

```
dotnet ef migrations add Feedback_Add --context RazorPagesMovieContext --output-dir
Migrations
dotnet ef database update --context RazorPagesMovieContext
```

Еквівалент у **Package Manager Console** Visual Studio:

```
Add-Migration Feedback_Add -Context RazorPagesMovieContext -OutputDir Migrations
Update-Database -Context RazorPagesMovieContext
```

Це альтернативні набори команд, а не дві послідовні частини роботи. Команди передбачають, що інструменти EF Core вже встановлені й проєкт збирається.

У лабораторній послідовно з'являються зміни для відповіді на відгук, самої таблиці звернень і полів відповіді на звернення. Кожну міграцію створюють після відповідних змін моделі.

Параметр контексту важливий, якщо застосунок має окремий контекст Identity. Нові звернення треба додавати до контексту предметних даних, використаного в роботі.

Перед оновленням БД перегляньте `Up()` — що додається, і `Down()` — як описане зворотне перетворення. Порожня міграція може означати неправильний контекст або відсутність нових змін порівняно зі знімком. Не видаляйте застосовані міграції чи базу навмання. Видалення останньої **ще не застосованої** міграції — інший випадок, ніж відкат уже зміненої БД.

6. Як завантажується список відгуків і звернень

Адміністратору потрібні не лише записи, а й пошук, сортування, ознака наявності відповіді.

```
var query = _context.Reviews.AsNoTracking().AsQueryable();

if (onlyUnreplied)
{
    query = query.Where(r => r.AdminReply == null || r.AdminReply == "");
}

var items = await query
    .OrderByDescending(r => r.CreatedAt)
    .ToListAsync();
```

Це спрощений варіант запиту, щоб побачити його будову:

1. Беремо джерело даних — відгуки.
2. За потреби додаємо умову: залишити лише ті, на які не відповіли.
3. Розташовуємо новіші записи першими.
4. Виконуємо запит і отримуємо список.

`r => r.CreatedAt` — лямбда-вираз: «для кожного запису `r` використати його дату». `Where` описує умову відбору. `OrderByDescending` — сортування за спаданням.

Поки будується `IQueryable`, ми переважно формуємо опис майбутнього запиту. `ToListAsync()` запускає його виконання й матеріалізує результат у список.

`AsNoTracking()` доречний для читання: EF Core не відстежуватиме зміни завантажених сутностей для подальшого збереження. Для редагування в цій роботі запис завантажується окремо, без цього виклику.

Навіщо Include і Select

`Include(r => r.Movie)` завантажує пов'язаний фільм разом із відгуком, коли потрібні самі сутності. Наприклад, у формі відповіді необхідна назва фільму.

`Select(...)` формує потрібний набір даних. У списках роботи використовується `RowVM`: один об'єкт для одного рядка таблиці. Якщо проєкція вже вибирає `r.Movie.Title`, EF Core може сформувати необхідний SQL без окремого `Include`; не варто вважати `Include` обов'язковим у кожному такому запиті.

`record RowVM(...)` — стислий спосіб оголосити тип для передавання даних. У цьому прикладі він не створює нову таблицю БД, а описує результат запиту для показу.

Пошук через GET

```
[BindProperty(SupportsGet = true)]
public string? q { get; set; }

[BindProperty(SupportsGet = true)]
public bool onlyUnreplied { get; set; }
```

Форма з `method="get"` формує адресу приблизно такого вигляду:

```
/Admin/Reviews?q=цікавий&onlyUnreplied=true
```

Цю адресу можна скопіювати й відкрити з такими самими фільтрами. GET підходить для пошуку, оскільки пошук не змінює дані.

У ручному checkbox з методички непозначений прапорець не передає значення; нова властивість `bool` лишається `false`. Зазначене `value="true"` задає значення для позначеного прапорця.

У запитах `EF.Functions.Like(..., "%текст%")` означає пошук входження тексту. Чутливість до регістру залежить від БД та її налаштувань, а `%` і `_` у шаблоні мають спеціальне значення.

7. Як відкривається форма відповіді

Посилання у списку:

```
<a asp-page="./Reply" asp-route-id="@it.ReviewId">Відповісти</a>
```

`./Reply` — сусідня сторінка. `asp-route-id` передає ідентифікатор обраного запису. На сторінці відповіді:

```
@page "{id:int}"
```

За стандартної маршрутизації після перенесення в `Admin` адреса може бути `/Admin/Reviews/Reply/7`. `7` — ключ відгуку, а `int` вимагає цілочисельного сегмента маршруту.

Далі ASP.NET Core викликає `OnGetAsync`. Обробник знаходить запис за ключем і готує дані для розмітки. Якщо запису немає, `NotFound()` повертає відповідь 404: правильний формат адреси не гарантує існування запису.

Що відбувається після натискання «Зберегти»

```
GET: відкрити форму → OnGetAsync → показати HTML
POST: надіслати форму → OnPostAsync → перевірити → зберегти
GET: відкрити список після перенаправлення
```

`OnGetAsync` і `OnPostAsync` можуть бути в **одному класі PageModel**. Це два обробники різних запитів до тієї самої сторінки. Браузер не викликає C#-метод напряму: він надсилає HTTP-запит, а платформа вибирає обробник.

Кожен запит зазвичай отримує новий екземпляр PageModel. Дані, завантажені під час GET, не зберігаються в цьому об'єкті до POST. Саме тому POST повторно читає запис із БД.

`async` і `await` дозволяють асинхронно очікувати операцію з БД або поштою. Це не означає «запустити й забути»: код після `await` виконується після завершення очікуваної операції.

8. Як asp-for і BindProperty працюють разом

```
[BindProperty]
public ReplyInput Input { get; set; } = new();
```

```
<textarea asp-for="Input.AdminReply"></textarea>
```

За стандартних налаштувань Tag Helper сформує HTML-поле приблизно такого вигляду:

```
<textarea id="Input_AdminReply" name="Input.AdminReply"></textarea>
```

`name` використовується для передавання значення на сервер. `id` потрібний для зв'язку з підписом, JavaScript і CSS.

`asp-for` допомагає **сформувати HTML**, а `[BindProperty]` дозволяє механізму model binding **заповнити властивість PageModel з отриманого запиту**. У цьому прикладі вони працюють на різних етапах одного процесу. Механізм model binding.

`[BindProperty]` не зберігає нічого в БД. Після прив'язування обробник ще має перевірити введення, змінити сутність і викликати `SaveChangesAsync()`.

Модель введення окрема від сутності не випадково. Користувачеві дозволено передати текст, але не довільно призначити автора відповіді чи її дату. Такі значення встановлює сервер.

9. Валідація: перевіряємо, а не просто приймаємо

```
[Required(ErrorMessage = "Введіть тему повідомлення.")]
[StringLength(120, ErrorMessage = "Тема може містити до 120 символів.")]
[Display(Name = "Тема")]
public string Subject { get; set; } = string.Empty;
```

- `Required` — значення обов'язкове.
- `StringLength` — обмежує довжину.
- `EmailAddress` — перевіряє форму email, але не існування скриньки.
- `Display` — задає зрозумілий підпис поля; це не перевірка.

`ModelState` містить результати прив'язування і перевірок. Наприклад, помилка виникне, якщо текст не можна перетворити на потрібне число або він перевищує дозволену довжину.

```
if (!ModelState.IsValid)
{
    return Page();
}
```

`Page()` повторно відображає поточну сторінку з помилками. Це не новий GET: `OnGetAsync()` автоматично не викликається. Якщо розмітка показує дані з БД, перед поверненням їх потрібно завантажити й присвоїти властивостям PageModel.

У прикладі відповіді на відгук у методичці бракує перевірки `ModelState.IsValid`. Її слід додати до збереження. Логіка фрагмента обробника може бути такою:

```
// Фрагмент OnPostAsync у сторінці відповіді на відгук.
var review = await _context.Reviews
    .Include(r => r.Movie)
    .FirstOrDefaultAsync(r => r.Id == id);

if (review is null)
    return NotFound();

if (!ModelState.IsValid)
{
    Review = review;
    MovieTitle = review.Movie?.Title ?? "";
```

```
    return Page();
}

// Лише після перевірки: змінити відповідь і зберегти запис.
```

Назва ключа в цьому фрагменті — `Id`; використовуйте фактичну властивість ключа вашого `Review`. Не перезаписуйте `Input` старими значеннями з БД у гілці помилки: користувач має побачити власне введення.

`asp-validation-for` показує помилки конкретного поля. `asp-validation-summary="ModelOnly"` призначений для загальних помилок моделі. `_ValidationScriptsPartial` підключає клієнтську перевірку, але JavaScript можна вимкнути, тому серверна перевірка залишається обов'язковою. Валідація моделей.

10. Контактна форма: від введення до запису Feedback

`Feedback` зберігає ім'я, email, тему, повідомлення, час створення та необов'язковий `UserId`.

Контактна сторінка доступна гостям через `[AllowAnonymous]`. Тому email у формі та обліковий запис — не те саме. Гість може вказати адресу без реєстрації; це саме по собі не доводить, що скринька йому належить.

Для користувача, який увійшов, `OnGetAsync()` попередньо заповнює ім'я та email через `UserManager<IdentityUser>`. Це зручність, а не заборона змінити доступні поля.

Після POST обробник:

1. Перевіряє `ModelState`.
2. Перевіряє антиспам-поле.
3. Створює `Feedback` з дозволених полів `Input`.
4. Бере `UserId` з поточного облікового запису, якщо користувач увійшов.
5. Додає сутність і зберігає зміни.
6. Перенаправляє на контактну сторінку з повідомленням про успіх.

```
_context.Feedbacks.Add(feedback);
await _context.SaveChangesAsync();
```

Перший рядок позначає об'єкт як новий у контексті. Другий виконує збереження в базу даних. На цьому етапі користувачу **ще не надсилається відповідь поштою** — адміністратор опрацює звернення пізніше.

Honeypot — проста пастка для ботів

Поле `Input.Website` в методичці винесене за видиму область сторінки. Людина має лишити його порожнім, а простий бот може заповнити всі поля.

Якщо поле заповнене, обробник показує умовний успіх, але не зберігає звернення. Це лише евристика: вона не зупинить усіх ботів, а автоматичне заповнення браузером інколи може завадити людині. Для реального сервісу потрібні додаткові заходи проти зловживань, зокрема обмеження частоти запитів.

TempData і перенаправлення

```
[TempData]
public string? OkMessage { get; set; }
```

`TempData` допомагає перенести коротке повідомлення між запитами. Звичайна властивість PageModel після перенаправлення не збереглася б. Значення TempData зазвичай видаляється після прочитання.

Схема POST → Redirect → GET зменшує ризик повторного надсилання форми при звичайному оновленні сторінки після успіху. Вона не гарантує захисту від швидкого подвійного натискання чи повторного запиту.

У сторінці відповіді на звернення встановлюється `TempData["Ok"]`, але в наведеній розмітці списку бракує його відображення. Над таблицею можна додати:

```
@if (TempData["Ok"] is string message)
{
    <div class="alert alert-success">@message</div>
}
```

11. Сервіс електронної пошти: навіщо інтерфейс

```
public interface IEmailSender
{
    Task SendAsync(string toEmail, string subject,
        string htmlBody, string? plainText = null);
}
```

Інтерфейс — це домовленість: будь-який клас, який реалізує цей інтерфейс, має надати відповідний метод. Він описує, **що можна зробити**, але не визначає конкретний спосіб надсилання.

У роботі два способи:

- `ConsoleEmailSender` показує лист у консолі для перевірки логіки. Реального листа немає.
- `SmtpEmailSender` передає лист SMTP-серверу.

PageModel залежить від `IEmailSender`, а не від конкретного SMTP-сервера. Тому для першого тестування не обов’язково налаштовувати справжню пошту.

Dependency Injection — передавання залежностей

```
builder.Services.AddTransient<IEmailSender, ConsoleEmailSender>();
```

Реєстрація в `Program.cs` означає: «коли потрібен `IEmailSender`, створи `ConsoleEmailSender`». Вона виконується до `builder.Build()`. Для SMTP замініть реалізацію в цій реєстрації; для навчального перемикання не треба залишати обидва варіанти активними.

Конструктор PageModel отримує сервіс:

```
public ReplyModel(RazorPagesMovieContext context,
    IEmailSender email,
    UserManager<IdentityUser> userManager)
{
    _context = context;
    _email = email;
    _userManager = userManager;
}
```

Платформа сама передає потрібні об'єкти з контейнера сервісів. `Transient` означає новий екземпляр при кожному отриманні сервісу з контейнера, а не при кожному виклику `SendAsync`.

У Identity теж існує інтерфейс із назвою `IEmailSender`. Якщо виникла неоднозначність, перевірте namespace: у цій роботі потрібний власний інтерфейс із `Services`.

12. SMTP і конфігурація пошти

SMTP — протокол передавання електронної пошти. Вебзастосунок передає лист поштовому серверу, а той далі працює з його доставкою.

Параметр	Значення
`Host`	Адреса SMTP-сервера
`Port`	Порт підключення
`User`, `Password`	Дані автентифікації в поштовому сервісі
`From`	Адреса відправника, дозволена сервісом
`FromName`	Видиме ім'я відправника
`UseSsl`	У прикладі керує `SmtpClient.EnableSsl`

`smtp.example.com` і `support@example.com` — демонстраційні значення, не готова конфігурація. Порт і спосіб захисту потрібно брати з документації обраного поштового провайдера.

```
builder.Services.Configure<EmailOptions>(
    builder.Configuration.GetSection("Email"));
```

Цей код пов’язує секцію конфігурації `Email` з властивостями `EmailOptions`. Через `IOptions<EmailOptions>` сервіс отримує налаштування; `.Value` повертає сам об’єкт із ними.

Справжній пароль не варто зберігати у файлі, який потрапить у Git чи архів лабораторної. Для локальної розробки використовуйте User Secrets, для розгортання — належне сховище секретів або налаштування середовища. User Secrets відокремлює секрети від проєкту, але не шифрує їх і не є виробничим сховищем. Зберігання секретів під час розробки.

`System.Net.Mail.SmtpClient` пояснює механіку прикладу, але Microsoft не рекомендує його для нової розробки. У нових реальних застосунках варто окремо розглянути сучасну поштову бібліотеку або API провайдера. Це не означає, що навчальний приклад обов’язково перестає працювати. Зауваження до SmtpClient.

13. Як надсилається відповідь на звернення

У формі задано:

```
<form method="post" asp-page-handler="Send">
```

Це вибирає **іменований обробник**:

```
public async Task<IActionResult> OnPostSendAsync()
```

У назві `OnPostSendAsync`: `Post` — HTTP-метод, `Send` — ім’я обробника, `Async` — асинхронний варіант. Параметр `handler=Send` за стандартної маршрутизації зазвичай передається в адресі форми.

Обробник завантажує звернення, перевіряє введення, формує лист, викликає `_email.SendAsync(...)`, а потім зберігає тему й текст відповіді, автора та час.

Адресата беруть зі збереженого звернення, а автора відповіді — з поточного облікового запису. Ці дані не потрібно довіряти прихованим полям HTML-форми.

Посилання `mailto:...` у таблиці працює інакше: відкриває поштову програму користувача. Воно не викликає ваш сервіс надсилання та не фіксує відповідь у БД.

HTML листа: текст потрібно кодувати

У Razor звичайний вивід `@Model.Message` HTML-кодується автоматично. Проте вручну сформований C#-рядок HTML-листа не проходить автоматично через цей захист.

У методичці для тіла відповіді використано `Input.Body.Replace("\n", "<br/>")`. Це зберігає переноси, але не кодує сам введений текст. Для текстової відповіді без дозволеного HTML безпечніше:

```
using System.Net;
```

```
var safeBody = WebUtility.HtmlEncode(Input.Body)
    .Replace("\r\n", "\n")
    .Replace("\n", "<br/>");

var htmlBody = $"<p>{safeBody}</p>";
```

Спочатку текст перетворюється на безпечний для HTML-вміст, потім додаються контрольовані теги переносу. Ім'я та тему при вставленні в HTML теж потрібно кодувати. Не застосовуйте `Html.Raw` до довільного тексту користувача. Принципи безпечного HTML-виведення.

Чому лист і запис у БД можуть розійтися

У прикладі спочатку надсилається лист, потім зберігається результат. Можливі ситуації:

- SMTP недоступний — надсилання завершується помилкою.
- Сервер прийняв лист, але збереження в БД не вдалося — лист уже переданий, хоча сайт цього не зафіксував.
- Використано `ConsoleEmailSender` — виклик успішний, але лист лише виведений у консоль.

Тому повідомлення «надіслано» в режимі консолі потрібно сприймати як імітацію. Навіть успішна передача SMTP-серверу не доводить, що адресат отримав і прочитав лист.

Для навчального варіанта важливо показати зрозумілу помилку замість хибного успіху та не втратити введення. У виробничих системах для узгодження БД і пошти застосовують черги й окремі записи про заплановане надсилання. Звичайна транзакція БД не може відкликати вже переданий лист.

14. Хто має право відкривати адміністративні сторінки

Автентифікація відповідає на запитання «хто увійшов?». **Авторизація** — «що цьому користувачу дозволено?».

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireAdministratorRole",
        policy => policy.RequireRole("Admin"));
});

builder.Services.AddRazorPages(options =>
{
    options.Conventions.AuthorizeFolder(
        "/Admin", "RequireAdministratorRole");
});
```

Це фрагмент конфігурації після перенесення адміністративних сторінок. Потрібно доповнювати наявні налаштування `Program.cs`, зберігаючи реєстрацію Identity, ролей, контекстів і необхідні middleware, а не замінювати файл цим уривком.

`Admin` — назва ролі. `RequireAdministratorRole` — назва політики, яка вимагає цю роль. `/Admin` — шлях відносно `Pages`, не `/Pages/Admin`.

До перенесення папок окремо захищаються `/Movies` і `/Reviews`. Сам захист `/Admin` не поширюється на сторінки, які фізично залишилися поруч із цією папкою. Авторизація папок Razor Pages.

Перевірка `User.IsInRole("Admin")` у `_Layout` лише приховує або показує меню. Без серверної авторизації користувач міг би ввести адміністративну адресу вручну.

Стандартний захист Razor Pages від підроблених POST-запитів також варто зберегти: Form Tag Helper додає antiforgery-токен у звичайну POST-форму, а платформа перевіряє його. Це окремий механізм від перевірки ролі та від антиспам-поля.

15. Layout, головна сторінка й український інтерфейс

`Pages/Shared/_Layout.cshtml` — спільний макет: навігація, підключення стилів і скриптів, загальна оболонка. `@RenderBody()` позначає місце для вмісту конкретної сторінки.

Головну сторінку змінюють у `Pages/Index.cshtml`, а загальне меню — у `_Layout.cshtml`. Якщо вставити великий вітальний блок у Layout, він може з'явитися на всіх сторінках.

Для тематичної головної сторінки достатньо чітко відповісти на три запитання: що це за каталог, що тут може зробити відвідувач і куди натиснути далі. Кнопки «Переглянути каталог» та «Зв'язатися з нами» корисніші за суто декоративний текст.

Українізація охоплює не лише заголовки, а й підписи форм, повідомлення про помилки, успішні операції, порожні списки, вхід і реєстрацію. `Display(Name = "...")` не перекладає автоматично всі повідомлення валідації чи інтерфейс Identity.

Для дат важливо розрізняти збереження та відображення. `DateTime.UtcNow` зберігає універсальний час. Показувати його можна в потрібному часовому поясі; `ToLocalTime()` на сервері використовує часовий пояс сервера, а не автоматично відвідувача.

Якщо адміністративне меню реалізоване як Bootstrap dropdown, потрібен відповідний JavaScript Bootstrap. Самі CSS-класи й `data-bs-toggle="dropdown"` не забезпечують розкриття меню без підключеного скрипту.

16. Перенесення сторінок у Pages/Admin

Переносити потрібно пари `.cshtml` і `.cshtml.cs` разом. Після перенесення перевірте:

1. Посилання: `/Movies/Index` стає `/Admin/Movies/Index`, `/Reviews/Index` — `/Admin/Reviews/Index`.
2. `namespace` класів і відповідні директиви `@model`, якщо ви змінюєте простори імен.
3. Правила захисту папок.
4. Перенаправлення, кнопки «Назад» та посилання з інших сторінок.

Відносне `./Reply` залишається правильним, якщо список і форма відповіді переїхали разом. Абсолютні посилання на стару папку потрібно оновити.

Саме перенесення Razor-сторінок не змінює модель даних, тому не вимагає міграції. Публічний каталог і контактну форму не слід випадково зробити адміністративними.

17. Як перевірити, що ви зрозуміли й виконали роботу

Перевірка	Очікуваний результат
Гість відкриває каталог і контакти	Сторінки доступні
Гість або звичайний користувач вводить адресу адмінсторінки	Немає доступу до керування; перенаправлення на вхід або відмова залежно від стану входу
Адміністратор зберігає відповідь на відгук	Вона видима біля відповідного відгуку в каталозі
Адміністратор очищує відповідь	Текст, автор і дата відповіді очищені
У форму введено надто довгий текст	Є помилка, дані не зберігаються
Контактна форма має неправильний email	З'являється повідомлення валідації
Гість надсилає коректне звернення	Запис є в адмінсписку; `UserId` може бути `null`
Заповнено honeypot	Нового звернення немає, хоча показано умовний успіх
Вибрано «Лише без відповіді»	Список відфільтрований
Відповідь на звернення через консольний сервіс	Лист показано в консолі, реального надсилання немає
Відповідь через налаштований тестовий SMTP	Лист видно в тестовій скриньці або сервісі-перехоплювачі
Після успішної форми оновлено сторінку	Звичайне оновлення GET не створює ще одного запису
Папки перенесені в Admin	Меню, форми й перенаправлення використовують правильні шляхи

Спочатку перевіряйте логіку з `ConsoleEmailSender` і вигаданими даними. Потім — з тестовим SMTP-середовищем. Не використовуйте адреси сторонніх людей для налагодження.

Запитання для самоперевірки

1. Чому відповідь на відгук і відповідь на звернення мають різні механізми?
2. Чим `Feedback` відрізняється від `ContactInput` і `IndexModel`?
3. Чому `[BindProperty]` не замінює `SaveChangesAsync()`?
4. Чому `return Page()` після помилки може вимагати повторного завантаження даних?
5. Як `asp-page-handler="Send"` пов'язаний з `OnPostSendAsync()`?

6. Чому пароль SMTP не можна передавати браузеру або зберігати в публічному репозиторії?
7. Чому приховане адміністративне меню ще не означає захищену сторінку?
8. Чому успішний виклик поштового сервісу не гарантує прочитання листа?

Підсумок

Центральна ідея роботи — керований шлях повідомлення: **прийняти введення → перевірити → зберегти → дати відповідь → показати результат**. Кожен крок має власний інструмент: Razor-форма, model binding, валідація, EF Core, сервіс пошти та авторизація. Розуміння цих меж допомагає не лише повторити код лабораторної, а й самостійно змінити його під власний застосунок.